

# RELATÓRIO TÉCNICO-CIENTÍFICO DE CALIBRAÇÃO EMPÍRICA

Engenharia de Programa 2026.2 / Prof. Diego Frias

Paradigma F.O.C.A. (Custo Estimado de Operações Primitivas)

Nome da Equipe (Fantasia): L<sup>2</sup>F.exe

Aluno 1: Lucas Oliveira Caldas (Responsável por  $\tau_a$ )  
Aluno 2: Felipe Mendonça (Responsável por  $\tau_c$ )  
Aluno 3: Luis Jackson (Responsável por  $\tau_o$ )

**Objetivo Central:** Executar a medição empírica, filtragem estatística de *outliers* e homologação dos custos de tempo ( $\tau_o, \tau_c, \tau_a$ ) e funções isoladas ( $F$ ), garantindo a estabilidade do ambiente ( $CV < 0.15$ ) para uso nas análises estáticas de complexidade.

## 1. ESPECIFICAÇÃO RIGOROSA DO AMBIENTE COMPUTACIONAL (SETUP)

*Regra: O grupo deverá eleger uma única máquina para rodar todos os experimentos. Medições realizadas em máquinas diferentes invalidarão a matriz F.O.C.A. da equipe.*

| Parâmetro Físico / Lógico         | Especificação Detalhada (Preenchimento Obrigatório) |
|-----------------------------------|-----------------------------------------------------|
| Processador (Modelo Exato)        | 12th Gen Intel(R) Core(TM) i7-12650HX (2.00 GHz)    |
| Frequência Base e Turbo Boost     | 2,00 GHz                                            |
| Arquitetura de Cache (L1, L2, L3) | L1: 1,2 MB L2: 11,5 MB L3: 24,0 MB                  |
| Memória RAM (Capacidade/Freq.)    | 16,0 GB                                             |
| Sistema Operacional e Kernel      | Windows 11 kernel 26200.9457                        |
| Plano de Energia do SO            | 381b4222-f694-41f0-9685-ff5bb260df2e (Equilibrado)  |
| Versão Exata da Linguagem         | Python 3.14.0                                       |

## 2. PARÂMETROS METODOLÓGICOS DO EXPERIMENTO

*Regra: Documentar os hiperparâmetros estatísticos e algorítmicos utilizados na captura.*

| Parâmetro de Captura                 | Valor | Justificativa Técnica |
|--------------------------------------|-------|-----------------------|
| Tamanho da Amostra Interna ( $n$ )   | 10000 |                       |
| Número de Repetições Globais ( $r$ ) | 1000  |                       |
| Estratégia de Filtro de Outliers     |       |                       |

## 3. PAINEL GRÁFICO DE ESTABILIDADE (EXIGÊNCIA VISUAL)

Para cada **macro-primitiva** avaliada, o responsável deverá anexar (no final deste documento) um painel contendo exatamente 3 gráficos gerados a partir do script de calibração:

1. **Gráfico de Dispersão Temporizada:** Eixo X (Iteração  $r$ ) vs Eixo Y (Tempo Acumulado/Médio). Evidenciar picos anômalos (SO / Garbage Collector).
2. **Histograma de Dados Brutos:** Frequência mostrando assimetria e caudas longas (Thermal Throttling / Cache Misses).
3. **Histograma Pós-Filtragem:** Distribuição após aplicação do filtro (Percentil ou  $\sigma$ ), com linhas de corte visíveis. Deve tender à normalidade.

#### 4. MATRIZES EXAUSTIVAS DE CALIBRAÇÃO ( $\tau$ )

Instrução: Preencham as matrizes convertendo os tempos brutos em filtrados. O cálculo do CV deve seguir a fórmula  $CV = \frac{\sigma}{\mu}$ . Apenas linhas com **CV Filtrado**  $< 0.15$  serão homologadas.

##### 4.1. Custo de Atribuições ( $\tau_a$ ) - Responsável: Aluno 1

| Operação e Tipagem Exata                                 | $\mu_{bruta}$ | $\sigma_{bruta}$ | $CV_{bruto}$ | $\mu_{fil}(\tau_a)$ | $\sigma_{fil}$ | $CV_{fil} < 0.15$ |
|----------------------------------------------------------|---------------|------------------|--------------|---------------------|----------------|-------------------|
| <code>var_int ← cte_int</code> (ex: <code>a = 5</code> ) | 7.93ns        | 2.90ns           | 0.3652       | 7.35ns              | 1.25ns         | 0.1700            |
| <code>var_float ← cte_float</code>                       | 16.16ns       | 6.45ns           | 0.3989       | 15.15ns             | 4.92ns         | 0.3249            |
| <code>var_bool ← cte_bool</code>                         | 8.50ns        | 2.41ns           | 0.2836       | 8.25ns              | 1.41ns         | 0.1715            |
| <code>var_int ← var_int</code> (ex: <code>a = b</code> ) | 8.20ns        | 2.82ns           | 0.3445       | 7.93ns              | 1.56ns         | 0.1969            |
| <code>var_float ← var_float</code>                       | 9.40ns        | 1.62ns           | 0.2480       | 9.04ns              | 1.62ns         | 0.1795            |

##### 4.2. Custo de Comparações ( $\tau_c$ ) - Responsável: Aluno 2

| Operação e Tipagem Exata                          | $\mu_{bruta}$ | $\sigma_{bruta}$ | $CV_{bruto}$ | $\mu_{fil}(\tau_c)$ | $\sigma_{fil}$ | $CV_{fil} < 0.15$ |
|---------------------------------------------------|---------------|------------------|--------------|---------------------|----------------|-------------------|
| Igualdade: <code>cte_int == cte_int</code>        | 11.87ns       | 2.24ns           | 0.1887       | 11.53ns             | 1.27ns         | 0.1105            |
| Igualdade: <code>var_int == cte_int</code>        | 22.30ns       | 6.15ns           | 0.2756       | 21.09ns             | 4.10ns         | 0.1945            |
| Igualdade: <code>var_int == var_int</code>        | 12.76ns       | 3.78ns           | 0.2961       | 12.45ns             | 3.01ns         | 0.2419            |
| Igualdade: <code>var_float == var_float</code>    | 10.04ns       | 3.30ns           | 0.3291       | 9.09ns              | 1.14ns         | 0.1252            |
| Desigualdade: <code>var_int != var_int</code>     | 12.71ns       | 3.36ns           | 0.2648       | 12.33ns             | 1.23ns         | 0.0996            |
| Relacional: <code>var_int &gt; var_int</code>     | 14.33ns       | 3.63ns           | 0.2532       | 14.20ns             | 3.31ns         | 0.2333            |
| Relacional: <code>var_float &lt; cte_float</code> | 19.01ns       | 8.36ns           | 0.4398       | 17.45ns             | 3.12ns         | 0.1786            |

##### 4.3. Custo de Operações Matemáticas ( $\tau_o$ ) - Responsável: Aluno 3

| Operação e Tipagem Exata                            | $\mu_{bruta}$ | $\sigma_{bruta}$ | $CV_{bruto}$ | $\mu_{fil}(\tau_o)$ | $\sigma_{fil}$ | $CV_{fil} < 0.15$ |
|-----------------------------------------------------|---------------|------------------|--------------|---------------------|----------------|-------------------|
| Soma Int: <code>var_int + cte_int</code>            | 10.37ns       | 1.22ns           | 0.1177       | 10.17ns             | 0.58ns         | 0.0571            |
| Soma Int: <code>var_int + var_int</code>            | 9.97ns        | 3.23ns           | 0.3240       | 9.54ns              | 0.79ns         | 0.0827            |
| Soma Float: <code>var_float + var_float</code>      | 11.65ns       | 1.11ns           | 0.0954       | 11.45ns             | 0.33ns         | 0.0291            |
| Subtração Int: <code>var_int - var_int</code>       | 11.02ns       | 3.18ns           | 0.2887       | 10.53ns             | 1.60ns         | 0.1518            |
| Multiplicação Int: <code>var_int * var_int</code>   | 9.05ns        | 1.43ns           | 0.1586       | 8.87ns              | 0.20ns         | 0.0225            |
| Mult. Float: <code>var_float * var_float</code>     | 11.82ns       | 1.67ns           | 0.1410       | 11.50ns             | 0.45ns         | 0.0388            |
| Divisão Float: <code>var_float / var_float</code>   | 14.99ns       | 3.82ns           | 0.2546       | 14.10ns             | 1.76ns         | 0.1249            |
| Divisão Int (Piso): <code>var_int // var_int</code> | 11.30ns       | 3.66ns           | 0.3241       | 10.75ns             | 0.31ns         | 0.0284            |
| Módulo (Resto): <code>var_int % var_int</code>      | 13.41ns       | 4.81ns           | 0.3585       | 12.69ns             | 1.17ns         | 0.0924            |

**4.4. Custo de Coleção de Funções Opacas ( $F$ ) - Responsável: Equipe (Opcional + 1 ponto extra)**

| Função (Biblioteca/Nativa)            | $\mu_{bruta}$ | $\sigma_{bruta}$ | $CV_{bruto}$ | $\mu_{fil}(\tau_f)$ | $\sigma_{fil}$ | $CV_{fil} < 0.15$ |
|---------------------------------------|---------------|------------------|--------------|---------------------|----------------|-------------------|
| Ex: <code>math.sqrt(var_float)</code> | 30.37ns       | 5.41ns           | 0.1782       | 29.13ns             | 0.90           | 0.0309            |
| Ex: <code>abs(var_int)</code>         | 14..09ns      | 1.70ns           | 0.1207       | 13.93ns             | 1.28           | 0.0916            |
| [Função 1]:                           | 14.42ns       | 2.05ns           | 0.1423       | 14.18ns             | 1.15           | 0.0810            |
| [Função 2]:                           | 163.25ns      | 20.04            | 0.1227       | 159.45ns            | 13,35          | 0.0837            |

## 5. ANÁLISE CRÍTICA DA ENGENHARIA F.O.C.A. (DEFESA DOS RESULTADOS)

### Questão 1 (Acesso à Memória):

Comparando os resultados da tabela  $\tau_a$ , explique a diferença de tempo (se houver) entre operar uma atribuição de um valor constante literal (`var ← cte`) contra a atribuição copiando outra variável residente (`var ← var`). Onde o gargalo arquitetural se encontra no segundo caso?

Com base na tabela  $\tau_a$ , a atribuição de inteiros com constante (7,35 ns) é mais rápida do que a cópia entre variáveis (7,93 ns), enquanto nos floats a cópia (9,04 ns) é mais rápida do que o uso de literais (15,15 ns) devido ao custo de carregar constantes flutuantes da memória. O gargalo arquitetural em `var ← var` é a latência de leitura da variável de origem na memória/cache (ou registradores) antes de efetuar a escrita, enquanto constantes inteiras já vêm embutidas na própria instrução de máquina (operando imediato).

### Questão 2 (ALU vs. FPU):

Ao cruzar os dados da tabela  $\tau_o$ , qual foi o impacto percentual no tempo de processamento ao realizar operações matemáticas puramente em Inteiros (`int`) contra as mesmas operações envolvendo Ponto Flutuante (`float`)? A arquitetura do processador eleito favorece algum desses cálculos?

Com base na tabela  $\tau_o$ , as operações em ponto flutuante (`float`) são, em média, 24,5% mais lentas do que em inteiros (`int`). Individualmente, a soma em `float` é 20,02% mais demorada (11,45 ns vs. 9,54 ns), a multiplicação é 29,65% mais lenta (11,50 ns vs. 8,87 ns) e a divisão é 31,16% mais lenta (14,10 ns vs. 10,75 ns). A arquitetura do processador favorece os cálculos em inteiros (ALU), pois a unidade de ponto flutuante (FPU) exige etapas adicionais de hardware – como alinhamento de mantissa e normalização de expoentes –, aumentando o tempo de execução.

### Questão 3 (Complexidade de Hardware):

Na tabela de operações matemáticas ( $\tau_o$ ), ordene da instrução de hardware mais rápida para a mais lenta: Soma, Multiplicação e Divisão (Exata/Resto). A diferença empírica reflete a complexidade eletrônica/lógica dos circuitos somadores e multiplicadores reais?

A ordenação das operações na tabela  $\tau_o$ , da mais rápida para a mais lenta ( $\mu_{fil}$ ), é: Multiplicação Int (8,87 ns), Soma Int (9,54 ns), Subtração Int (10,53 ns), Divisão Int (10,75 ns), Soma Float (11,45 ns), Multiplicação Float (11,50 ns), Módulo (12,69 ns) e Divisão Float (14,10 ns). Este resultado reflete apenas parcialmente a complexidade dos circuitos reais: confirma que a Divisão e os cálculos em Float são mais lentos devido à maior complexidade algorítmica e etapas como normalização e IEEE 754, mas apresenta uma anomalia ao registrar a Multiplicação Int como mais rápida que a Soma Int, o que contradiz a teoria de hardware e decorre de otimizações do compilador ou do pipeline do processador.

### Questão 4 (Interferência Sistêmica e Filtragem):

Expliquem detalhadamente quais métricas melhoraram após a aplicação do filtro de *outliers* (Percentil ou Desvio Padrão). Na visão de engenharia de software da equipe, quais eventos físicos da máquina ou lógicos do SO geraram as anomalias nas pontas do Histograma Bruto?

### Questão 5 (Benchmarking e Desempenho no Pior Caso - *FLOPS/IPS*):

Analisando todas as matrizes homologadas (estritamente as com  $CV < 0.15$ ), identifiquem a primitiva validada mais lenta de todo o experimento (maior tempo  $\mu_{fil}$ ). Utilizando esse tempo de pior cenário, calculem o desempenho mínimo garantido do processador da equipe em FLOPS (*Floating-Point Operations Per Second*) ou IPS (*Instructions Per Second*), através da relação  $1/\tau_{pior}$ . Apresentem o cálculo e discutam brevemente como este *benchmark* empírico de pior caso se compara com a frequência teórica (GHz) anunciada pelo fabricante.